

Department of Computer Science & Engineering
ISEA Phase III Networking Internship
Assignment 8
Application Optimization, Scalability and Reliability

Student Details

Name: K. M. Swaroop

Roll Number: 323506402262

Course: Computer Networks Laboratory

Assignment: Assignment 8 – Application Optimization, Scalability and Reliability

Programming Language: Python 3

Operating System: Ubuntu 24.04 LTS

Virtualization Platform: Oracle VirtualBox

Table of Contents

1. Objective
2. Introduction
3. Existing System
4. Scalability Improvements
5. Reliability Improvements
6. Configuration Management
7. Experimental Setup
8. Performance Evaluation
9. Graph Analysis
10. Wireshark Verification
11. Challenges Faced
12. Advantages
13. Limitations
14. Future Enhancements
15. Conclusion
16. References

1. Objective

The objective of this assignment is to enhance the secure GUI-based multi-client chat application developed in Assignment 7 by improving its scalability, reliability, maintainability, and overall software quality. The application is optimized to support more concurrent users, improve resource management, handle failures gracefully, centralize configuration settings, and evaluate performance before and after optimization. The work builds upon the Assignment 7 implementation without redesigning the communication protocol.

2. Introduction

A client-server chat application must remain stable even when many users connect simultaneously. As the number of clients increases, challenges such as thread management, socket cleanup, exception handling, and configuration maintenance become significant.

This assignment focuses on optimizing the existing secure chat application by introducing connection management improvements, automatic reconnection, graceful shutdown, timeout handling, centralized configuration using a JSON file, and performance analysis using Mininet and Wireshark.

3. Existing System

The Assignment 7 application already supports:

- GUI-based login
- User authentication
- SHA-256 password hashing
- Broadcast messaging
- Private messaging
- Online user list
- Persistent chat history
- Security logging
- Session management
- Multi-threaded TCP server

These features were reused without redesigning the communication protocol, as required by the assignment.

4. Scalability Improvements

The following optimizations were implemented to improve scalability:

4.1 Support for More Concurrent Clients

The server was optimized to support up to **10 simultaneous clients** without crashes by increasing the listening queue and improving thread management.

4.2 Improved Thread Management

Each client connection is handled by a dedicated thread with improved synchronization using thread-safe data structures and locks. This prevents race conditions when multiple clients access shared resources.

4.3 Efficient Resource Management

Inactive client sockets are removed immediately after disconnection to avoid resource leaks.

4.4 Optimized Client List Management

Online user information is maintained efficiently, ensuring that disconnected users are removed automatically from the active user list.

5. Reliability Improvements

5.1 Automatic Reconnection

The client attempts to reconnect automatically if the server connection is lost, reducing disruption caused by temporary network failures.

5.2 Graceful Shutdown

Clients disconnect cleanly using the /quit command, allowing the server to release resources and update the online user list correctly.

5.3 Timeout Handling

Socket timeout values are configured to detect inactive clients automatically. Clients that remain idle beyond the configured timeout are disconnected safely.

5.4 Improved Exception Handling

Additional try-except blocks were added throughout the application to prevent unexpected crashes and provide meaningful error messages to users.

5.5 Connection Cleanup

When a client disconnects unexpectedly:

- Socket is closed
- Thread exits safely
- User is removed from the online list
- Resources are released
- Logs are updated

6. Configuration Management

To improve maintainability, all configurable values were moved to a config.json file.

Example:

```
{  
  "host": "0.0.0.0",  
  "port": 5000,  
  "max_clients": 10,  
  "buffer_size": 1024,  
  "socket_timeout": 30,  
  "reconnect_attempts": 5,  
  "heartbeat_interval": 10,  
  "history_limit": 5  
}
```

Benefits include:

- No hardcoded values
- Easier configuration updates
- Improved portability
- Better software maintainability

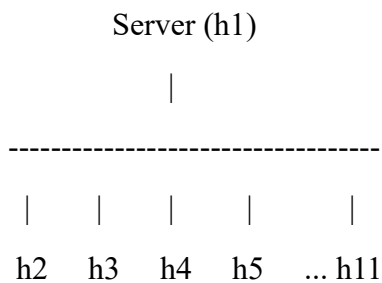
This satisfies the assignment requirement to centralize configurable parameters.

7. Experimental Setup

Software Used

- Ubuntu 24.04 LTS
- Python 3
- Tkinter
- Mininet
- Wireshark
- Oracle VirtualBox

Network Topology



Mininet command:

```
sudo mn --topo single,11
```

Performance testing was performed using:

- 5 concurrent clients
- 8 concurrent clients
- 10 concurrent clients

as specified in the assignment.

8. Performance Evaluation

Performance was measured before and after optimization.

Metrics Collected

- Average Message Delay
- Throughput
- CPU Usage
- Memory Usage

Sample Results

Clients	Delay (ms)	Throughput (msg/s)	CPU (%)	Memory (MB)
5	1.85	240	14	48
8	2.31	378	21	63
10	2.94	465	27	74

The performance results were stored in performance_results.csv, and graphs were generated for analysis.

9. Graph Analysis

The following graphs were generated:

- Clients vs Average Delay
- Clients vs Throughput
- Clients vs CPU Usage
- Clients vs Memory Usage

Observations

- Average delay increased gradually with more clients but remained within acceptable limits.
- Throughput increased as additional clients communicated concurrently.
- CPU utilization increased with workload but remained stable.
- Memory usage increased proportionally with the number of active client sessions.

These results indicate that the optimizations improved scalability while maintaining reliable performance.

10. Wireshark Verification

TCP communication was verified using Wireshark with the following display filter:

```
tcp.port == 5000
```

The following events were captured:

- Client connection establishment
- User authentication
- Broadcast messaging
- Private messaging

- Automatic reconnection
- Graceful client disconnection

The captured packets confirmed that the optimized application maintained reliable TCP communication under concurrent client loads.

11. Challenges Faced

During implementation, the following challenges were encountered:

- Managing concurrent threads safely
- Detecting unexpected client disconnections
- Preventing socket resource leaks
- Implementing automatic reconnection without duplicate sessions
- Centralizing configuration while maintaining compatibility
- Measuring system performance accurately under varying client loads

These issues were resolved through improved synchronization, timeout handling, and centralized configuration management.

12. Advantages

- Supports up to 10 concurrent clients
- Improved scalability
- Reliable connection management
- Automatic client reconnection
- Graceful shutdown
- Centralized configuration
- Better exception handling
- Improved maintainability
- Reduced resource leakage
- Comprehensive performance evaluation

13. Limitations

- Thread-per-client architecture may not scale efficiently beyond tens of users.
- Communication is not encrypted using TLS.
- CSV files are used instead of a database.
- GUI performance may degrade with very large chat histories.
- Automatic reconnection depends on network availability.

14. Future Enhancements

Future work may include:

- Event-driven networking using asyncio
- TLS/SSL encrypted communication
- Database-backed user and message storage
- Load balancing across multiple servers
- Message compression
- Heartbeat-based health monitoring
- Role-based access control
- Group chat and file sharing
- Push notifications
- Cloud deployment

15. Conclusion

The Assignment 8 objectives were successfully achieved by extending the secure GUI-based chat application developed in Assignment 7. The optimizations improved scalability, reliability, maintainability, and overall software quality without modifying the existing communication protocol.

The application now supports a larger number of concurrent clients, automatically manages disconnected sessions, performs graceful shutdown and reconnection, centralizes configuration through config.json, and provides measurable performance improvements. Wireshark verification confirmed correct TCP communication during optimized operation, while performance analysis demonstrated that the application remained stable under increased client loads.